```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)
```

```
# Load the Placement dataset from the specified file path into a Pandas Data
df = pd.read_csv("/content/disease_prediction.csv")
```

```
# Display the first five rows of the DataFrame to quickly inspect the data
df.head()
```

	patient_id	age	gender	glucose_mg_dl	cholesterol_mg_dl	systolic_bp	di
0	1	32	Male	101	235	152	
1	2	31	Male	124	191	134	
2	3	45	Male	57	141	114	
3	4	75	Female	69	268	120	
4	5	53	Male	107	163	131	

```
print(df.columns)
```

```
Index(['patient_id', 'age', 'gender', 'glucose_mg_dl', 'cholesterol_mg_dl',
       'systolic_bp', 'diastolic_bp', 'bmi', 'heart_rate', 'smoking',
       'alcohol_consumption', 'physical_activity', 'family_history',
       'disease'],
      dtype='object')
```

```
df = df.drop(columns=[
    'patient_id',
    'gender',
    'diastolic_bp',
    'heart_rate',
    'smoking',
    'alcohol_consumption',
```

```
    'physical_activity',
    'family_history'
])
```

```
print(df.head())
print(df.columns)
```

```
   age  glucose_mg_dl  cholesterol_mg_dl  systolic_bp   bmi disease
0   32            101                235          152  28.5     Yes
1   31            124                191          134  33.9     Yes
2   45             57                141          114  27.2      No
3   75             69                268          120  21.5     Yes
4   53            107                163          131  23.3     Yes
Index(['age', 'glucose_mg_dl', 'cholesterol_mg_dl', 'systolic_bp', 'bmi',
       'disease'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   age                1000 non-null   int64
 1   glucose_mg_dl      1000 non-null   int64
 2   cholesterol_mg_dl  1000 non-null   int64
 3   systolic_bp        1000 non-null   int64
 4   bmi                1000 non-null   float64
 5   disease            1000 non-null   object
dtypes: float64(1), int64(4), object(1)
memory usage: 47.0+ KB
```

```
print(df.isnull().sum())
```

```
age                 0
glucose_mg_dl       0
cholesterol_mg_dl   0
systolic_bp         0
bmi                 0
disease             0
dtype: int64
```

```
X = df[['age',
        'glucose_mg_dl',
        'cholesterol_mg_dl',
        'systolic_bp',
        'bmi']]

y = df['disease']
```

```
print(X.head())
print(y.head())
```

```
   age  glucose_mg_dl  cholesterol_mg_dl  systolic_bp   bmi
0   32            101                235          152  28.5
1   31            124                191          134  33.9
2   45             57                141          114  27.2
3   75             69                268          120  21.5
4   53            107                163          131  23.3
0    Yes
1    Yes
2     No
3    Yes
4    Yes
Name: disease, dtype: object
```

```
print(y.unique())
print(y.value_counts())
```

```
['Yes' 'No']
disease
Yes    501
No     499
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
model = LogisticRegression()
```

```
model.fit(X_train_scaled, y_train)
```

▾ LogisticRegression

```
LogisticRegression()
```

```
y_pred = model.predict(X_test_scaled)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.705
```

```
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[74 26]
 [33 67]]
```

```
import pickle
```

```
pickle.dump(model,open('HeartDisease.pkl','wb'))
```

```
pickle.dump(scaler, open('scaler.pkl', 'wb'))
```

```
import pickle

model = pickle.load(open("HeartDisease.pkl", "rb"))

test_data = [[65, 180, 280, 170, 35.0]]

prediction = model.predict(test_data)

print("Prediction:", prediction)
print("Probability:", model.predict_proba(test_data))
```

```
Prediction: ['Yes']
Probability: [[0. 1.]]
```

```
test_data = [[30, 85, 160, 110, 21.5]]

print(model.predict(test_data))
print(model.predict_proba(test_data))
```

```
['Yes']
[[0. 1.]]
```

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

    <title>Heart Disease Prediction</title>
```

```
<style>

    * {
        box-sizing: border-box;
    }

    body {
        margin: 0;
        font-family: Arial, sans-serif;
        background: linear-gradient(
            135deg,
            #e3f2fd,
            #fce4ec
        );

        min-height: 100vh;

        display: flex;
        justify-content: center;
        align-items: center;
    }

    .container {
        width: 450px;

        background: white;

        padding: 35px;

        border-radius: 15px;

        box-shadow:
            0 8px 25px rgba(0, 0, 0, 0.15);
    }

    h1 {
        text-align: center;

        color: #222;

        margin-bottom: 8px;
    }

    .subtitle {
        text-align: center;

        color: #666;

        font-size: 14px;

        margin-bottom: 25px;
    }

    .form-group {
```

```
        margin-bottom: 18px;
    }

    label {
        display: block;

        margin-bottom: 7px;

        font-weight: bold;

        color: #333;
    }

    .unit {
        font-weight: normal;

        color: #777;

        font-size: 13px;
    }

    input {
        width: 100%;

        padding: 12px;

        border: 1px solid #ccc;

        border-radius: 7px;

        font-size: 15px;

        outline: none;
    }

    input:focus {
        border-color: #007bff;

        box-shadow:
            0 0 5px rgba(0, 123, 255, 0.25);
    }

    input::placeholder {
        color: #aaa;
    }

    button {
        width: 100%;

        padding: 13px;

        margin-top: 10px;

        background: #007bff;
```

```
        color: white;

        border: none;

        border-radius: 7px;

        cursor: pointer;

        font-size: 16px;

        font-weight: bold;
    }

    button:hover {
        background: #0056b3;
    }

    .result {
        margin-top: 25px;

        padding: 15px;

        text-align: center;

        border-radius: 8px;

        background: #f1f8ff;

        color: #0056b3;

        font-size: 18px;

        font-weight: bold;
    }

    .info {
        margin-top: 20px;

        font-size: 12px;

        color: #777;

        text-align: center;

        line-height: 1.5;
    }

    @media (max-width: 500px) {

        .container {
            width: 90%;

            padding: 25px;
        }
```

```
            }

        </style>

    </head>


    <body>

    <div class="container">

        <h1>❤️ Heart Disease Prediction</h1>

        <p class="subtitle">
            Enter the patient's health information
        </p>


        <form action="/predict" method="POST">


            <!-- Age -->

            <div class="form-group">

                <label>
                    Age
                    <span class="unit">(years)</span>
                </label>

                <input
                    type="number"
                    name="age"
                    placeholder="Example: 45"
                    min="1"
                    max="120"
                    step="1"
                    required
                >

            </div>


            <!-- Glucose -->

            <div class="form-group">

                <label>
                    Blood Glucose
                    <span class="unit">(mg/dL)</span>
                </label>

                <input
                    type="number"
                    name="glucose"
```

```
                placeholder="Example: 100"
                min="0"
                step="0.1"
                required
            >

        </div>


        <!-- Cholesterol -->

        <div class="form-group">

            <label>
                Cholesterol
                <span class="unit">(mg/dL)</span>
            </label>

            <input
                type="number"
                name="cholesterol"
                placeholder="Example: 200"
                min="0"
                step="0.1"
                required
            >

        </div>


        <!-- Systolic BP -->

        <div class="form-group">

            <label>
                Systolic Blood Pressure
                <span class="unit">(mmHg)</span>
            </label>

            <input
                type="number"
                name="systolic_bp"
                placeholder="Example: 120"
                min="50"
                max="250"
                step="1"
                required
            >

        </div>


        <!-- BMI -->

        <div class="form-group">
```

```
            <label>
                Body Mass Index
                <span class="unit">(BMI)</span>
            </label>

            <input
                type="number"
                name="bmi"
                placeholder="Example: 24.5"
                min="5"
                max="80"
                step="0.1"
                required
            >

        </div>


        <!-- Button -->

        <button type="submit">
            🔍 Predict Disease
        </button>

    </form>


    <!-- Prediction -->

    {% if prediction %}

        <div class="result">
            {{ prediction }}
        </div>

    {% endif %}


    <div class="info">



    </div>

</div>

</body>

</html>
```

```
from flask import Flask, render_template, request
import pickle

app = Flask(__name__)
```

```
app = Flask(__name__)

# Load trained Logistic Regression model
model = pickle.load(open('HeartDisease.pkl', 'rb'))

# Load StandardScaler
scaler = pickle.load(open('scaler.pkl', 'rb'))


@app.route('/')
def home():
    return render_template('index.html')


@app.route('/predict', methods=['POST'])
def predict():

    # Get values from the form
    age = float(request.form['age'])
    glucose = float(request.form['glucose'])
    cholesterol = float(request.form['cholesterol'])
    systolic_bp = float(request.form['systolic_bp'])
    bmi = float(request.form['bmi'])

    # Keep the SAME order used during model training
    features = [[
        age,
        glucose,
        cholesterol,
        systolic_bp,
        bmi
    ]]

    # Scale the input using the saved scaler
    features_scaled = scaler.transform(features)

    # Make prediction
    prediction = model.predict(features_scaled)[0]

    # Display result
    if prediction == 'Yes':
        result = "❤️ Disease Detected"
    else:
        result = "✅ No Disease Detected"

    return render_template(
        'index.html',
        prediction=result
    )


if __name__ == '__main__':
    app.run(debug=True)
```